

BE-6B Notification Engine

Event-Driven Notification Engine with Multi-Channel Delivery

Project Presentation

Tawhidul Islam • BE-6B • Backend / System Engineering

Problem & Objectives

The problem

Applications generate many notification events, but delivery needs validation, routing, compliance checks, retries, provider failover, and tracking.

Core objectives

Process events reliably; support multiple channels; enforce user preferences and compliance; track delivery; provide operational visibility.

Engineering focus

Event-driven architecture, durable persistence, fast-access state, reliability patterns, automated testing, Docker, and CI/CD.

System Architecture

Backend-first architecture with distinct event, persistence, delivery, and reliability responsibilities



Event backbone

Kafka is the primary notification event stream, supporting durable events, consumer groups, partition-based parallelism, and replay.

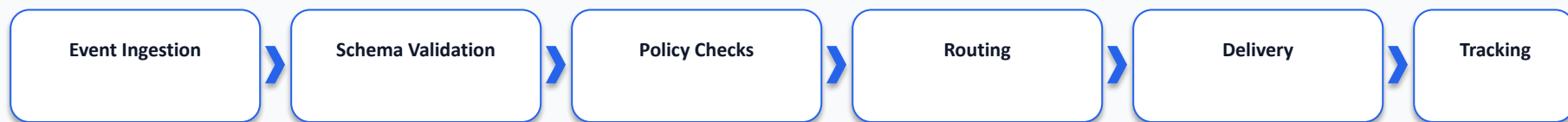
Persistence

PostgreSQL is the durable system of record; Redis provides low-latency cache and transient state.

Supporting queues

RabbitMQ handles supporting asynchronous workloads where queue semantics are preferable.

Event-Driven Flow



Validate

Incoming events are checked against the supported event taxonomy and payload rules.

Apply policy

Preferences, consent, DND/quiet hours, and frequency caps are evaluated.

Track outcome

Delivery results are persisted for tracking, retries, analytics, or DLQ processing.

Notification Processing

1. Validate

Zod-based event and payload validation ensures supported event types and required fields are present.

2. Decide

Notification rules evaluate preferences, consent, DND, quiet hours, and frequency caps.

3. Prepare

Template rendering, personalization, localization, and channel-specific formatting prepare the message.

4. Deliver

Routing and provider selection initiate delivery; state changes are recorded for tracking.

Multi-Channel Delivery

Email

Provider integration with delivery acknowledgement and failure handling.

SMS

Channel-specific formatting and provider handling.

Push

Push notification delivery through the provider abstraction.

WhatsApp

WhatsApp delivery through the provider layer.

In-App

In-app delivery and realtime support through Socket.IO.

Compliance & Preferences

User preferences

Channel preferences and routing adapters allow notification behavior to reflect user choices.

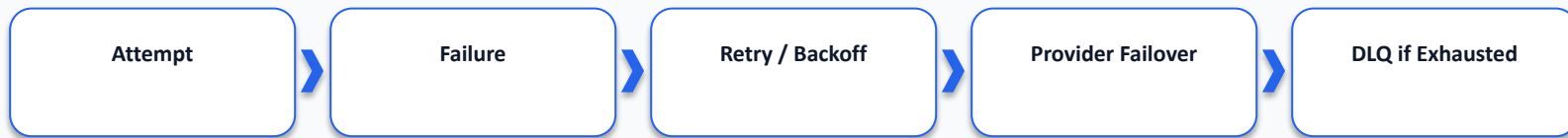
DND / Quiet Hours

Notifications can be classified, queued, or bypassed according to do-not-disturb and quiet-hours rules.

Frequency Caps

High-frequency notifications can be limited using fast-access state and policy checks.

Reliability: Retry / Failover / DLQ



Retry

Retry policy, retry budget, scheduler, and worker components handle transient failures.

Circuit Breaker

Provider health and circuit-breaker behavior prevent repeated calls to unhealthy providers.

Dead-Letter Queue

Failed messages can be classified and moved to DLQ processing for inspection and recovery.

Database & Infrastructure

PostgreSQL 15

Primary durable data store for users, notifications, state history, preferences, templates, providers, consent, and DLQ records.

Redis 7

Low-latency cache and transient state for rate limiting, frequency-cap related state, and analytics-related counters.

Kafka + RabbitMQ

Kafka provides the primary event backbone; RabbitMQ supports queue-oriented asynchronous workloads.

Docker

Docker Compose provides the local multi-service environment with app, PostgreSQL, Redis, Kafka, and RabbitMQ.

Testing & CI/CD

Automated Tests

Vitest + Supertest cover event validation, APIs, delivery providers, compliance, retries, DLQ, analytics, and end-to-end behavior.

Quality Gates

TypeScript typecheck, ESLint, Prettier, production build, tests with coverage, and repository checks.

GitHub Actions

CI runs linting, tests, coverage, security/build checks, and infrastructure-aware verification using Node 22.

Results / Verification

449 / 449 tests

Final local verification reported 449 passing tests across 51 test files.

~89% coverage

- ≈89.51% statements
- ≈81.14% branches
- ≈89.78% functions
- ≈89.47% lines.

CI + Docker

Typecheck, lint, format, build, Docker Compose, and GitHub Actions verification completed successfully.

Conclusion & Future Scope

What was delivered

A backend notification engine with event-driven processing, multi-channel delivery, compliance controls, reliability mechanisms, persistence, analytics, testing, Docker, and CI/CD.

Demo scope

A lightweight web frontend can act as a demonstration client for the deployed backend without becoming the primary BE-6B implementation.

Future scope

Production provider credentials, richer operational dashboards, deeper load testing, expanded deployment automation, and additional frontend capabilities.